SENTİNEL

Mars Rover Sensör Veri İşleme Gösterge Paneli

Kabul Testi Teknik Raporu | 2026

YÖNETİCİ ÖZETİ

Bu rapor, SENTİNEL uygulamasının Playwright tabanlı uçtan uca (E2E) kabul test altyapısını kapsamlı biçimde belgelemektedir. Test süiti, 15 test dosyası ve yaklaşık 1.000 UI testi ile uygulamanın tüm panel rotalarını, viewport uyumluluğunu, navigasyon senaryolarını ve API uç noktalarını doğrulamaktadır.

Testler, Chromium tarayıcısı üzerinde çalışmakta; Vite geliştirme sunucusu ile tam entegrasyon sağlanmaktadır. CI ortamında otomatik yeniden deneme ve sıkı mod etkinleştirilmiştir.

1. Proje Kapsamı ve Amaç

SENTİNEL kabul test süiti, uygulamanın aşağıdaki katmanlarını sistematik olarak doğrulamak amacıyla tasarlanmıştır:

- Frontend yükleme ve render doğruluğu (Landing + Dashboard shell)
- 12 panel rotasının her birinde HTTP yanıtı, DOM yapısı ve içerik bütünlüğü
- Mobil, tablet ve masaüstü viewport uyumluluğu (8 farklı çözünürlük)
- Yan menü navigasyon akışları (132 panel-çifti kombinasyonu)
- Geçersiz ve bilinmeyen URL segmentleri için yönlendirme davranışı
- Backend FastAPI sağlık ve dokümantasyon uç noktaları (isteğe bağlı)

2. Teknik Altyapı

2.1 Teknoloji Yığını

Bileşen	Detay
Test Çerçevesi	Playwright (TypeScript)
Tarayıcı	Chromium
Frontend	Vite + React (port 5173, strictPort: true)
Backend	FastAPI (port 8000, isteğe bağlı)
CI/CD	forbidOnly: true, retries: 2
Varsayılan Worker	1 (sıralı); PW_PARALLEL=1 ile çok çekirdek

2.2 Proje Dizin Yapısı

```
acceptance-test/
├── playwright.config.ts   # webServer, baseURL, workers
├── page-objects/           # PO sınıfları
│   ├── base-page.ts
│   ├── sentinel-landing.page.ts
│   └── sentinel-dashboard.page.ts
└── tests/
    ├── constants.ts        # DASHBOARD_ROUTES, URL'ler
    └── *.spec.ts           # 15 senaryo dosyası
```

3. Test Dosyaları ve Kapsam Matrisi

Aşağıdaki tablo, süitteki tüm test dosyalarını, yaklaşık test sayılarını ve kapsadıkları senaryoları özetlemektedir. Kesin sayı için npx playwright test --list komutu kullanılabilir.

Test Dosyası	Test Sayısı	Açıklama
smoke.spec.ts	~2	Temel duman testi: / yüklenir, gösterge paneli kabuğu görünür
landing.spec.ts	~10	Landing metinleri, navigasyon linkleri, CTA'lar, harici NASA linki
matrix-landing.spec.ts	~15	/ için geniş matris: GET, lang=tr, hero/marquee/footer, erişilebilirlik
dashboard-shell.spec.ts	~10	Gösterge kabuğu bileşenleri: sürüm, birim, rover, metrikler, footer
matrix-dashboard.spec.ts	204	Her panel rotası için 17 kontrol × 12 panel
matrix-dashboard-shell-extra.spec.ts	180	Kabuk detayları: aside, 12 menü, footer etiketleri, logo, scroll
matrix-viewport.spec.ts	39	Mobil/tablet/geniş viewport'ta tüm paneller
matrix-viewport-extra.spec.ts	65	Ek çözünürlükler: xs, iPhone, iPad mini, FHD, WQHD
navigation.spec.ts	~24	Doğrudan URL ve yan menü ile her panele navigasyon
navigation-pairs.spec.ts	132	Her panelden diğer her panele geçiş: 12 × 11
panels.spec.ts	~12	Her panelde ana içeriğin doğrulanması
routing.spec.ts	~5	Bilinmeyen segmentler → gösterge; logo → /; catch-all → landing
invalid-segments.spec.ts	200	NAV dışı tek segmentlerin yönlendirme doğrulaması
catchall-paths.spec.ts	100	2 ve 3 segmentli bilinmeyen yolların / 'e düşmesi
api-optional.spec.ts	2	GET /health ve /docs (FastAPI) — varsayılan: atlanır

Toplam: ~1.000 UI testi (API testleri kapalıyken). Kesin sayı için: `npx playwright test --list`

4. Sayfa Nesnesi (Page Object) Mimarisi

Test süiti, bakımı kolaylaştırmak için Page Object Model (POM) tasarım deseni üzerine kurulmuştur. Üç temel sınıf, tüm DOM etkileşimlerini kapsüllemektedir:

Sınıf	Sorumluluk
SentinelLandingPage	Ana sayfa (/): marka bileşenleri ve panele geçiş kontrolleri
SentinelDashboardPage	Gösterge kabuğu: logo, yan panel, footer, metrik widget'ları
BasePage	Ortak yardımcı metodlar: bekleme, gezinme, ekran görüntüsü

Bu mimari sayesinde, UI değişikliklerinin yalnızca ilgili sayfa nesnesi güncellenerek tüm testlere yansıtılması mümkündür.

5. Ortam Değişkenleri ve Yapılandırma

Değişken	Açıklama
PW_PARALLEL=1	Paralel çalıştırma (worker sayısı CPU çekirdeğine göre, fullyParallel açık)
RUN_BACKEND_API_TESTS=1	api-optional.spec.ts içindeki istekleri etkinleştirir; aksi halde testler atlanır
BACKEND_URL	Opsiyonel. Varsayılan: http://127.0.0.1:8000/health (constants.ts üzerinden)

Backend API testleri için örnek çalıştırma:

```
$env:RUN_BACKEND_API_TESTS="1"
npx playwright test tests/api-optional.spec.ts
```

6. Kurulum ve Çalıştırma

6.1 Önkoşullar

- Node.js 18 veya üzeri
- Depo kökünde frontend/ ve acceptance-test/ klasörlerinin yan yana bulunması
- Port 5173'ün boş olması (Vite strictPort: true ile çalışır)

6.2 Kurulum Adımları

1. Frontend bağımlılıklarını yükleyin:

```
cd frontend && npm install
```

2. Test bağımlılıklarını ve Playwright tarayıcılarını yükleyin:

```
cd ..\acceptance-test
npm install
npx playwright install
```

6.3 Testleri Çalıştırma

Komut	Açıklama
npm test	Tüm testleri çalıştır (tek worker, sıralı)
npm run test:ui	Playwright UI modu — adım adım izleme
npx playwright test --list	Tüm test adlarını listele
npx playwright test tests/smoke.spec.ts	Yalnızca tek dosyayı çalıştır
npm run typecheck	TypeScript tip kontrolü

7. CI/CD Entegrasyonu

playwright.config.ts içindeki CI ortamı yapılandırması aşağıdaki özellikleri etkinleştirir:

- forbidOnly: true — .only ile işaretli testlerin CI'a girmesini engeller
- retries: 2 — her başarısız test için en fazla iki yeniden deneme
- reuseExistingServer: false — CI'da her çalışmada temiz sunucu başlatılır
- webServer — Playwright, Vite dev sunucusunu otomatik başlatıp hazır hale gelmesini bekler

8. Sorun Giderme

Sorun	Çözüm
Sunucu açılmıyor	frontend/ içinde npm run dev elle çalışıyor mu kontrol edin. resolveFrontendDir() bir üst dizini arar.
Port 5173 meşgul	Çakışan süreci kapatın veya boşaltın (strictPort: true devre dışı bırakmaz).
Testler çok yavaş	PW_PARALLEL=1 ile paralel mod deneyin (CPU/RAM yükü artar).
Backend testi başarısız	Sunucu 127.0.0.1:8000 değilse BACKEND_URL değişkenini ayarlayın.

SONUÇ

SENTİNEL kabul test süiti, yaklaşık 1.000 UI testiyle uygulamanın fonksiyonel doğruluğunu, viewport uyumluluğunu ve navigasyon bütünlüğünü kapsamlı biçimde güvence altına almaktadır. Page Object Model mimarisi, uzun vadeli bakım kolaylığı sağlarken CI entegrasyonu sürekli kalite kontrolünü mümkün kılmaktadır.